CS102 - Week 3 Notes Outline and Study Guide

Table of Contents

What is inside

- Week 3 overview and master framework: algorithms, ordered steps, counting work, algorithmic complexity, Big O, comparing approaches, and safe input handling.
- Lesson 1 - Welcome to Algorithms
- Lesson 2 - Why We Need Algorithms in Real Life
- Lesson 3 - Turning Everyday Problems Into Step-by-Step Solutions
- Lesson 4 - Challenge: Algorithms
- Lesson 5 - Counting Work: How Many Steps Does an Algorithm Take
- Lesson 6 - Complexity: What It Means and Why It Matters
- Lesson 7 - Introduce the Big O Notation
- Lesson 8 - Analyzing Algorithms with Big O
- Lesson 9 - Challenge: Algorithm Analysis
- Lesson 10 - Comparing Two Solutions to the Same Problem
- Lesson 11 - Safe Algorithms: Handling Edge Cases and Bad Input
- Lesson 12 - Challenge: Advanced Algorithm Analysis
- Review tools, glossary, common mistakes, trace practice, and cheat sheet

Week 3 at a Glance

Week 3 is where algorithms stop sounding like a fancy computer science word and start becoming a way to think clearly. The week begins with the simplest idea: an algorithm is an ordered set of steps for solving a problem. Then it keeps raising the standard. Good algorithms are not just steps; they are clear, repeatable, precise, testable, and useful when the input gets bigger.

The early lessons use everyday examples because your brain already understands algorithms before you ever write code. Ordering pizza, brushing your teeth, washing clothes, checking a bank balance, making a sandwich, and sending a text message all involve a goal, a starting point, and steps that must happen in a sensible order. The course uses those familiar patterns to build the coding habit underneath them.

The second half of the week will move from writing steps to counting work. That is the bridge into complexity and Big O. Instead of only asking whether code gives the right answer, you begin asking how much work the code does as the problem grows. This is the beginning of algorithm analysis: comparing approaches by how they scale, not just whether they technically work once.

Memory hook: clear steps first, then count the work, then compare growth.

Week 3 Master Framework

Skill	What it means	Why it matters
Algorithm thinking	Break a task into clear ordered steps.	Turns vague goals into something a computer or person can follow.
Pseudocode thinking	Write instructions in plain English before Python.	Lets you solve the logic before wrestling syntax goblins.
Counting work	Track how many times a step runs.	Shows why more input often means more runtime.
Complexity thinking	Describe how work grows as input grows.	Helps compare solutions that are both technically correct.
Safe algorithm thinking	Plan for empty input, missing values, and weird cases.	Prevents crashes and makes code reliable.

Made with Goodnotes

Lesson 1 - Welcome to Algorithms

Big Idea

An algorithm is a clear set of ordered steps used to solve a problem. The important word is ordered. A random pile of advice is not an algorithm. A picture of the result is not an algorithm. A vague goal like "make pizza" is not an algorithm. The algorithm is the specific path from the starting point to the finished result.

Full Teaching Explanation

This lesson introduces algorithms in the friendliest way possible: by showing that you already use them constantly. When you order pizza for friends, you do not simply shout "pizza" into the universe and hope dinner materializes. You follow a sequence: count how many people are eating, ask what toppings they want, choose the size and number of pizzas, call or order from the store, and pay.

That ordered list matters because each step depends on earlier information. You cannot choose the right number of pizzas before you know how many people are eating. You cannot order accurately before you know toppings. You cannot pay before the order exists. The lesson is training you to see that solving problems is often about arranging the work in the right order.

In programming, an algorithm is the plan behind the code. Python syntax is how you express the plan, but the algorithm is the thinking underneath it. Two students can write different Python code and still use the same algorithm. They may name variables differently or format the code differently, but if the steps and decisions are the same, the algorithm is basically the same.

The key beginner mistake is thinking algorithms are automatically complicated. They are not. A simple recipe, a checkout process, or a set of directions can be an algorithm. Later, algorithms can become much more formal, but the foundation is still the same: a goal, an ordered process, and a way to know when the task is finished.

Examples and Mental Models

Situation	Is it an algorithm?	Why
Ordering pizza	Yes	There is a goal and a clear order of steps from count people to pay.
A photo of pizza	No	It shows the result, but not instructions for reaching it.
A friend saying "I like pizza"	No	It is information, not a procedure.
A numbered recipe	Yes	It tells you what to do first, next, and last.
A vague goal like "get dinner"	Not yet	It needs precise steps before it becomes an algorithm.

Out Loud Programmer Thinking

When I see a problem, I should ask: What is the goal? What information do I need first? What step must happen before another step? What decision points exist? What tells me I am done?

This habit matters because computers do not understand vague intention. They only follow instructions. The more precise the algorithm, the easier it becomes to turn the plan into code.

Made with Goodnotes

Pseudocode Example - Ordering Pizza

```
# Goal: order enough pizza for a group
count_people
ask_topping_choices
choose_pizza_sizes
choose_number_of_pizzas
place_order
pay_for_order
wait_for_delivery
```

Line-by-line explanation: the first line names the goal, and each later line is one action in the plan. This is pseudocode because it is not meant to run in Python yet. It is meant to make the algorithm clear before coding.

Terms, Definitions, and Why They Are Called That

- Algorithm: a step-by-step process for solving a problem. It is called an algorithm after historical mathematical procedures for calculation, but in beginner terms you can think of it as a reliable recipe for solving a task.
- Step: one action inside an algorithm. It is called a step because it moves you forward from where you are toward the result.
- Order: the sequence in which steps happen. Order matters because changing the order can change the result or break the process.
- Procedure: a repeatable set of instructions. It is useful when the same task must be done more than once.
- Pseudocode: plain-English code-like planning. It is called pseudo-code because it resembles code but is not actual executable code.

Common Mistakes and Helpful Tips

- Mistake: thinking an algorithm must be written in Python. The algorithm is the plan; code is one way to express it.
- Mistake: writing the goal instead of the steps. "Make a sandwich" is not enough; the algorithm explains how.
- Mistake: forgetting order. "Pay, choose toppings, call the store" is technically a list, but it is not a useful algorithm.
- Tip: use verbs at the start of steps: count, ask, choose, call, pay, print, check, return.
- Memory hook: an algorithm is not the wish. It is the route.

Notes

Made with Goodnotes

Lesson 2 - Why We Need Algorithms in Real Life

Big Idea

Algorithms matter because real-life tasks only become reusable, teachable, and programmable when they are broken into clear steps. A human can often infer missing steps, but a computer cannot. If the directions are vague, incomplete, or out of order, the result becomes unreliable.

Full Teaching Explanation

This lesson pushes the algorithm idea beyond pizza and into normal daily routines. Imagine trying to teach a robot how to brush its teeth. You cannot say, "just brush your teeth," because the robot has no hidden common sense. You have to describe the task in steps: go to the sink, pick up the toothbrush, put toothpaste on the brush, turn on the water, brush, rinse, and put everything back.

That example is funny because humans skip these details automatically. We use background knowledge. We understand that toothpaste belongs on the brush and that the brush goes in your mouth, not in your ear. Computers do not get that luxury. In programming, every important action must be represented somehow. Algorithms are how we make invisible human knowledge visible.

The lesson also uses examples like washing clothes, giving a weekly injection, and checking a bank balance on your phone. Each task has a starting point, a goal, and a repeatable process. If you do the steps in a different order, the result may be wrong or unsafe. For example, with laundry, you do not start the dryer before the clothes have been washed. With checking a bank balance, you do not check the balance before unlocking the phone and opening the app.

This is why algorithms help programming. Programs are built out of procedures. A login process, checkout flow, search feature, photo upload, bank transfer, medication reminder, or order tracker all require a sequence of precise actions. When you learn to describe real-life tasks algorithmically, you are learning to design code before you type code.

Everyday Algorithm Example - Checking a Bank Balance

```
# Goal: check a bank balance on a phone
unlock_phone
open_banking_app
enter_pin_or_use_face_id
tap_account
read_current_balance
close_app
```

This is not Python yet. It is an algorithm-style outline. Each line is a specific action, and the order matters. You cannot read the balance before the banking app is open and authenticated.

Everyday Algorithm Detector

Question	Why it matters
Does the task have a goal?	Algorithms are used to reach a result, not merely describe something.
Can the task be repeated?	A repeatable task can be written as a reusable procedure.
Does order matter?	If changing the order changes the outcome, algorithm thinking matters.
Are there hidden assumptions?	Those assumptions may need to become explicit steps.
Could a computer follow it?	The instructions must be specific enough for execution.

Made with Goodnotes

Terms, Definitions, and Why They Are Called That

- Instruction: a specific command telling someone or something what to do. It is called an instruction because it directs action.
- Sequence: steps arranged in order. It is called a sequence because one step follows another.
- Input: information the process needs before it can work. In a bank example, the input might be your PIN or login credentials.
- Output: the result produced by the process. In the bank example, the output is the account balance.
- Repeatable process: a task that can be done the same way again. This matters because code is usually written to handle repeated tasks, not one-time guesses.

Real English vs Programmer Thinking

Real English	Programmer Thinking
"Check my bank balance."	What app opens? What authentication is required? What account is selected? What output should appear?
"Do laundry."	What goes first: sort, add soap, choose cycle, start washer, move to dryer, fold? What can go wrong?
"Give an injection."	What supplies are needed? What safety checks happen before, during, and after the process?
"Brush teeth."	What tiny steps does a robot need because it has no common sense?

Common Mistakes and Helpful Tips

- Mistake: assuming obvious human steps are obvious to the computer. They are not.
- Mistake: writing steps that are too large, like "prepare medicine" instead of naming what preparation requires.
- Mistake: ignoring the starting condition. A process can fail if the phone is locked, the washer is full, or supplies are missing.
- Tip: ask, "Could a very literal robot follow this?" If not, add more detail.
- Tip: if a step contains the word "and," it may need to be split into two separate steps.
- Memory hook: real-life algorithms reveal the hidden instructions behind everyday tasks.

Notes

Made with Goodnotes

Lesson 3 - Turning Everyday Problems Into Step-by-Step Solutions

Big Idea

This lesson turns real-life tasks into algorithmic thinking by focusing on one clean rule: one action per line. When steps are too broad or smashed together, the algorithm becomes harder to follow and harder to translate into code.

Full Teaching Explanation

Lesson 3 takes the idea of algorithms and makes it more practical. It asks you to take an everyday task, like making hot chocolate, making a sandwich, or sending a text message, and break it into clear, ordered instructions. This is the same skill you will use when turning a programming problem into pseudocode.

The hot chocolate example starts with a vague goal: make hot chocolate. A better algorithm begins with concrete steps: get a mug, pour milk into the mug, heat the milk, add chocolate, stir, and serve. The sandwich example gets even more detailed. Instead of saying "make a sandwich," the algorithm says to get a plate, get bread, gather ingredients, put two slices of bread on the plate, apply condiments, add fillings, put the bread together, and eat.

The important idea is that each line should do one small thing. If a line says "get condiments, get meat, get cheese, and assemble the sandwich," it is doing too much. A programmer reading that line has to mentally unpack it. A computer would need those actions separated anyway. Good algorithms reduce that unpacking work by making the steps visible.

This is also where pseudocode becomes useful. Pseudocode is not about perfect Python syntax. It is about making the plan readable. Before you write a loop, condition, function, or data structure, you can sketch the behavior in plain English. That sketch helps you see missing steps, bad order, repeated work, and edge cases before the code gets involved and starts flinging syntax errors like confetti.

Messy Step vs Clean Step

Messy version	Cleaner algorithm version
Make a sandwich.	Get bread -> add filling -> close sandwich -> eat.
Get stuff and send the text.	Unlock phone -> open messages -> choose contact -> type message -> send.
Prepare hot chocolate.	Get mug -> pour milk -> heat milk -> add chocolate -> stir.
Fix the problem.	Identify error -> choose solution -> test solution -> verify result.

```
tap_create_message
choose_friend_name
type_message
review_message
press_send
```

Line-by-line explanation: every line starts with an action verb and represents one clear action. If the task were being turned into code, each line could eventually become a function call, a user action, or a step in a larger workflow.

Made with Goodnotes

Python-Flavored Example - Store Steps in a List

```
steps = [
    "get bread",
    "put cheese on bread",
    "add ham",
    "eat sandwich"
]

for step in steps:
    print(step)
```

This example shows the connection between algorithms and code. The list stores the ordered steps. The loop processes each step one at a time. The algorithm is still the sequence; Python is just helping display or execute it.

Line-by-Line Explanation

- steps = [...] creates an ordered list of instructions.
- Each string inside the list is one action.
- for step in steps: moves through the algorithm one instruction at a time.
- print(step) displays the current instruction.
- Real English translation: "Go through the recipe and say each step in order."
- Out loud programmer thinking: "If I can describe the steps clearly, I can start turning them into code."

Terms, Definitions, and Why They Are Called That

- Pseudocode: a plain-English version of code logic. It is called pseudo-code because it looks code-like but is not strict Python.
- Granularity: how large or small each step is. Fine-grained steps are small and specific; coarse-grained steps are broad.
- Action verb: a verb that tells the reader what to do, such as get, open, choose, type, check, or print.
- Sequential order: an order where steps happen one after another. It matters because later steps often depend on earlier ones.
- Trace: to walk through steps in order and observe what happens. It is called tracing because you follow the path of execution.

Common Mistakes and Helpful Tips

- Mistake: combining several actions into one line. Split "get bread and add cheese" into separate steps if the details matter.
- Mistake: starting in the middle of the task. Always define the starting condition.
- Mistake: writing steps out of order. A good test is to ask whether a literal robot could follow the instructions without guessing.
- Tip: use one verb per step whenever possible.
- Tip: after writing steps, trace them out loud and look for missing actions.
- Memory hook: if the summary makes you say "wait, how?", the algorithm needs more steps.

Notes

Made with Goodnotes

Lesson 4 - Challenge: Algorithms

Big Idea

This challenge lesson checks whether you can recognize an algorithm when you see one. An algorithm is not just a topic label, a random list, or a vague goal. It is a clear, ordered set of steps that can be followed to solve a problem or complete a task.

Full Teaching Explanation

The point of this lesson is to strengthen your algorithm instinct before the course gets more technical. The screenshots walk through everyday examples like laundry, making a sandwich, sending a text, boiling water for tea, brushing your teeth, and helping a friend make a peanut butter and jelly sandwich. Those examples may look simple, but they train the same thinking used in code: break the task into ordered instructions that a person or computer could follow.

A strong algorithm has three important qualities. First, it is ordered. If the steps happen in the wrong order, the result can fail. You cannot press send before typing the message, and you cannot start the washing machine before putting clothes and soap inside. Second, each step is actionable. A good step tells the doer what to do, not just what the final goal is. Third, the steps are clear enough that someone else could follow them without guessing.

This is why the lesson keeps pushing you toward pseudocode style: numbered steps, verb-first instructions, and one clear action per line. Pseudocode is not Python yet. It is a planning language that lets you prove the logic makes sense before you worry about exact syntax.

Algorithm or Not? Everyday Challenge Examples

Example	Is it an algorithm?	Why
Put clothes in, add soap, choose cycle, start, wait, rinse, finish	Yes	It is ordered, step-by-step, and tells exactly what to do.
A picture of a sandwich	No	It shows the goal, but it does not give instructions.
TEXT written on paper	No	It is a word, not a procedure.
Open chat, choose friend, type message, press send	Yes	It gives a repeatable process for sending a text.
Brushing teeth is good for you	No	That is a statement, not a sequence of steps.

Pseudocode Example - Making a Sandwich

```
1. Get bread.
2. Put cheese on the bread.
3. Add ham.
4. Eat the sandwich.
```

Line-by-Line / Real English Translation

This is intentionally plain English, but it behaves like an algorithm because each line is a small action and the steps happen in order. The sandwich example also shows why vague or combined steps can be a problem. A line like 'make the sandwich' is too broad because it hides the work inside one giant step.

Real English translation: start with the materials, perform one action at a time, then reach the final result. Out loud programmer thinking: 'Can another person follow this without asking what I meant?'

Terms, Definitions, and Why They Are Called That

Made with Goodnotes

- **Algorithm:** a clear set of steps for solving a problem. It is called an algorithm because it names the procedure, not just the answer.
- **Pseudocode:** plain-English planning code. It is called pseudo-code because it looks code-like but is not meant to run in Python.
- **Instruction:** one action the doer can perform. In algorithms, instructions should usually start with a verb such as get, open, choose, type, add, or print.
- **Sequence:** the order in which steps happen. A sequence matters when changing the order changes the result.
- **Ambiguity:** wording that can be interpreted multiple ways. Ambiguous instructions create bugs because the computer cannot guess what you meant.

Common Mistakes and Helpful Tips

- **Mistake:** writing the goal instead of the steps, like "clean the room" or "make sandwich." The goal is not the algorithm; the algorithm is how to get there.
- **Mistake:** putting too many actions on one line. If a tiny robot could get confused, split the step.
- **Mistake:** forgetting order. Some tasks, like texting or doing laundry, only work if the setup steps happen before the action step.
- **Tip:** read the algorithm out loud. If it sounds like a real set of instructions, you are close.
- **Memory hook:** clear steps, correct order, one action at a time.

Notes

Made with Goodnotes

Lesson 5 - Counting Work: How Many Steps Does an Algorithm Take?

Big Idea

This lesson introduces the idea that algorithms do work, and we can count that work. The main pattern is simple: if a loop runs once for each item in a collection, then more items means more loop steps.

Full Teaching Explanation

The lesson starts with a postcard or stamping example: if you have 3 postcards and do 2 stamping moves for each postcard, the total work grows as the number of postcards grows. Then it moves into code with a list of names. The computer goes through the list one name at a time. Each loop pass is one unit of work for the repeated part of the algorithm.

This is a major bridge between normal coding and algorithm analysis. Beginners often count only the lines of code they see. But computers do not run a loop line just once. A loop body runs once per item. If the list has 3 names, the body runs 3 times. If the list has 6 names, the same body runs 6 times. The code did not get longer, but the amount of work changed because the input got bigger.

That idea matters because real programs rarely process only 3 cute example items. They process photos, contacts, orders, messages, users, records, or search results. A tiny loop can be harmless with a small input and slow with a huge input. Counting work helps you predict how an algorithm will behave before it becomes a problem.

Code Example 1 - Counting One Step Per Name

```
names = ["Ana", "Ben", "Leo"]
steps = 0

for name in names:
    steps = steps + 1
    print(name)

print("Total steps:", steps)
```

Line-by-Line / Real English Translation

Line-by-line explanation: **names** holds the input collection. **steps = 0** starts the counter before any work happens. **for name in names:** loops once per item. Inside the loop, **steps = steps + 1** records one more pass through the repeated work.

Real English translation: 'For every name in the list, count one step and print the name.' With Ana, Ben, and Leo, the loop runs 3 times, so the final step count is 3.

How the Step Count Changes

Input size	Loop body runs	Final step count	What it teaches
3 names	3 times	3	One pass per item.
6 names	6 times	6	Doubling the list doubled the loop work.
5 numbers	5 times	5	A single loop over a list grows with the list.
8 items	8 times	8	The number of loop steps follows the input size.

Made with Goodnotes

Code Example 2 - Same Algorithm, Bigger Input

```
names = ["Ana", "Ben", "Leo", "Mia", "Ray", "Zoe"]
steps = 0

for name in names:
    steps = steps + 1
    print(name)

print("Total steps:", steps)
```

Line-by-Line / Real English Translation

The instructions are the same as the earlier code. The only thing that changed is the input list. Because the loop still runs once for each item, 6 names means 6 passes through the loop body. This is the key idea: the work grows when the input grows.

Out loud programmer thinking: 'I am not counting how many lines are written. I am counting how many times the repeated part runs.'

Terms, Definitions, and Why They Are Called That

- **Step count:** the number of basic actions an algorithm performs. It is called step count because we are counting the work step-by-step.
- **Input size:** how much data the algorithm receives, such as the number of names, numbers, photos, or records. It matters because bigger input often creates more work.
- **Loop iteration:** one complete pass through a loop body. It is called iteration because the loop repeats the same pattern again and again.
- **Linear growth:** work that grows in direct proportion to input size. If the list doubles and the work doubles, that is linear growth.
- **Counter variable:** a variable used to track how many times something happened, like **steps** in the examples.

Common Mistakes and Debug Thinking

- **Mistake:** counting the loop code line only once. The loop header appears once, but the loop body runs once per item.
- **Mistake:** forgetting to set the counter to 0 before the loop starts.
- **Mistake:** assuming all algorithms grow the same way. Some grow slowly, some grow quickly, and some explode with bigger inputs.
- **Debug tip:** print the counter inside the loop if you want to see it grow one pass at a time.
- **Memory hook:** number of items in the list = number of loop steps for a simple one-pass loop.

Notes

Made with Goodnotes

Code Example:

Python

Run | Submit code

```python
names = ["Ana", "Ben", "Leo"]
steps = 0

for name in names:
    steps = steps + 1
    print(name)

print("Total steps:", steps)
```

Output

```
Ana
Ben
Leo
Total steps: 3
```

Python

Run | Submit code

```python
names = ["Ana", "Ben", "Leo", "Mia", "Ray", "Zoe"]
steps = 0

for name in names:
    steps = steps + 1
    print(name)

print("Total steps:", steps)
```

Output

```
Ana
Ben
Leo
Mia
Ray
Zoe
Total steps: 6
```

Made with Goodnotes

Lesson 6 - Complexity: What It Means and Why It Matters

Big Idea

Complexity describes how the amount of work an algorithm does grows as the problem gets bigger. Two algorithms can both be correct, but one may stay fast as the input grows while the other becomes painfully slow.

Full Teaching Explanation

This lesson takes the step-count idea from Lesson 5 and zooms out. Instead of asking, 'How many steps does this exact tiny example take?' complexity asks, 'What happens when the problem gets bigger?' That is the question that matters in real programs, because your code may handle 5 contacts, 500 photos, or 50,000 records depending on the situation.

The lesson uses a contact-list story. One approach checks each name once until it finds the person. Another approach keeps re-checking pairs of contacts or doing extra repeated comparisons. Both approaches may eventually find the right answer, but the second one does far more unnecessary work as the list grows. With a few contacts, both feel fine. With thousands of contacts, the difference becomes obvious.

This is the heart of algorithmic complexity. Correctness tells us whether the algorithm gets the right answer. Complexity tells us how the work grows while getting that answer. A program can be correct and still feel slow, laggy, or frustrating if the algorithm does too much work for the input size.

One Scan vs Many Re-Checks

Number of contacts	Fast method: check each once	Slow method: compare many pairs	What happens
5	5 checks	10 checks	Both feel fast.
50	50 checks	1,225 checks	The slow method already does much more work.
500	500 checks	124,750 checks	The slow method starts to feel heavy.
5,000	5,000 checks	12,497,500 checks	The slow method explodes as the input grows.

Pseudocode Comparison - Same Goal, Different Work

```
# Approach A: one scan
for contact in contacts:
    if contact == target:
        return contact

# Approach B: repeated re-checking
for contact in contacts:
    for other_contact in contacts:
        compare(contact, other_contact)
```

Line-by-Line / Real English Translation

Approach A has one loop over the contacts. If the list gets bigger, the work grows in a direct, predictable way. Approach B has a loop inside another loop, so each contact can cause many more checks. That nested repeated work grows much faster.

Real English translation: 'Your way' checks each contact once. The slow way keeps comparing contacts with other contacts, even when those extra comparisons may not help. Both can be correct, but one wastes far more work as the list grows.

Why Complexity Matters in Real Life

Made with Goodnotes

Complexity is why some apps feel snappy and others feel like they are thinking about their life choices. A game might work fine with a few players, then lag when there are many players. A messaging app might search quickly through a small chat history but drag through years of messages. A photo app might load one album quickly but struggle when scanning thousands of images.

The user experience is often the visible symptom of algorithmic complexity. You do not always see the loop, scan, search, or comparison happening behind the scenes. You just feel the result: fast load, slow search, lag, freezing, or delay.

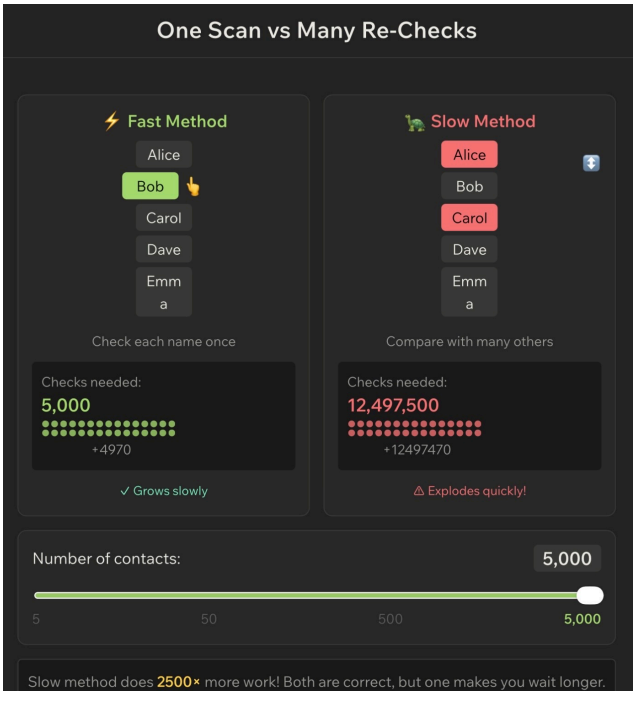
Terms, Definitions, and Why They Are Called That

- **Algorithmic complexity:** how the amount of work grows as the problem gets bigger. It is called complexity because it describes the growth behavior, not just one exact step count.
- **Growth rate:** the pattern of how work increases. If input doubles and work doubles, the growth is gentle; if input doubles and work quadruples, it grows much faster.
- **Scalable:** able to handle bigger input without becoming unreasonable. Scalable code keeps working well as data grows.
- **Efficient:** doing the needed job without wasting unnecessary work. Efficiency is about using fewer steps, time, or resources for the same result.
- **Nested loop:** a loop inside another loop. Nested loops often create much faster-growing work because repeated work happens inside repeated work.

Common Mistakes and Helpful Tips

- **Mistake:** thinking correct code is automatically good enough. Correct code can still be too slow for large inputs.
- **Mistake:** focusing only on exact seconds. Complexity cares more about the pattern of growth than one measured time on one machine.
- **Mistake:** ignoring input size. A solution that feels fine with 5 items may fall apart with 5,000 items.
- **Tip:** ask, "When the input grows, does the work grow slowly, directly, or explosively?"
- **Memory hook:** complexity = how work grows when the problem grows.

Notes



Made with Goodnotes

Lesson 7 - Introduce the Big O Notation

Big Idea

Big O notation is a way to describe the **shape of growth** in an algorithm. It does not try to predict the exact number of seconds a program will take. Instead, it asks: as the input gets bigger, how does the amount of work grow?

This lesson connects the earlier idea of counting steps to a cleaner language programmers use for comparison. Once you can say that one algorithm is O(n) and another is O(n^2), you can explain why both may work on small data, but one becomes painful when the data gets larger.

Full Teaching Explanation

Big O is useful because exact timing is noisy. One computer may run faster than another, one phone may be older, and one app may be busy with background work. If we only measure seconds, the answer changes depending on the device. Big O ignores those machine-specific details and focuses on the pattern: does the work stay about the same, grow one step per item, grow as pairs of items, or explode much faster?

The letter **n** usually means the input size. If you are searching a list of contacts, n is the number of contacts. If you are looping through photos, n is the number of photos. If you are comparing every item with every other item, the number of checks may grow as n times n, which becomes O(n^2).

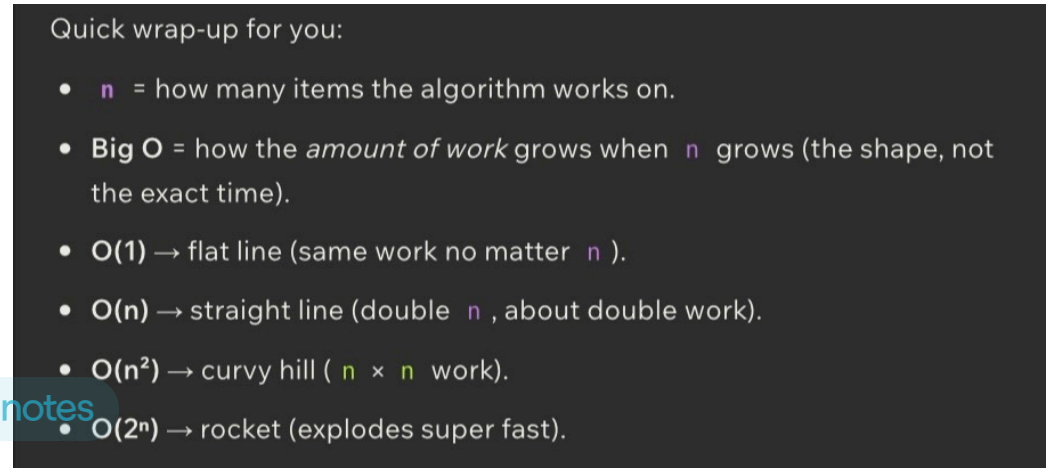
A helpful mental model is a growth meter. If adding more input barely changes the work, the algorithm is close to constant time. If doubling the input roughly doubles the work, it is linear. If doubling the input makes the work around four times larger, you are probably looking at quadratic growth. Big O is the vocabulary for naming those growth patterns.

Types of Measurement

Type	Meaning	Example
Best case	The easiest possible input. The algorithm gets lucky.	The first contact you check is the one you need.
Worst case	The hardest normal input. The algorithm has to do the most work.	The contact is last, or not found at all.
Average case	The expected middle-ground behavior across many normal inputs.	Sometimes the contact is near the front, sometimes near the end.

Core Big O Shapes

Big O	Called	Plain-English Meaning	Tiny Example
O(1)	constant time	Same amount of work no matter how many items exist.	Read the first item in a list; turn on a light.
O(log n)	logarithmic time	Work shrinks by cutting the problem down, often in half.	Binary search in a sorted list.
O(n)	linear time	Work grows one step for each item.	Loop through every name once.
O(n log n)	linearithmic time	Work grows a little faster than linear because items are split and recombined.	Many efficient sorting algorithms.
O(n^2)	quadratic time	Work grows as pairs of items, often from nested loops.	Compare every person with every other person.
O(2^n)	exponential time	Work doubles with each added choice.	Trying every yes/no combination.



Made with Goodnotes

Code Example 1 - Constant Time O(1)

```
# Same number of steps no matter how large the list is.
names = ["Ana", "Ben", "Leo", "Mia"]
first_name = names[0]
print(first_name)
```

Line-by-line explanation: the list could hold 4 names or 4 million names, but this code still asks for index 0 one time. The amount of work does not grow with n, so the growth class is O(1).

Code Example 2 - Linear Time O(n)

```
# One pass through the list.
names = ["Ana", "Ben", "Leo", "Mia"]
for name in names:
    print(name)
```

Line-by-line explanation: the loop body runs once per name. If there are 4 names, it runs 4 times. If there are 400 names, it runs 400 times. The work grows in a straight-line pattern with n, so this is O(n).

Code Example 3 - Quadratic Time O(n^2)

```
# Every item is compared with every other item.
names = ["Ana", "Ben", "Leo"]
for first in names:
    for second in names:
        print(first, second)
```

Line-by-line explanation: the outer loop runs n times, and for each outer loop, the inner loop also runs n times. That gives n * n total combinations. This is why nested loops over the same collection often mean O(n^2).

Terms, Definitions, and Why They Are Called That

- **Big O notation**: a notation for describing how algorithmic work grows. It is called Big O because it describes the order of growth, not the exact count.
- **n**: the input size. It is usually the number of items the algorithm must handle.
- **Constant time**: work that stays the same size. It is called constant because it does not change when n changes.
- **Linear time**: work that grows in a straight-line relationship with n. Double the items, roughly double the work.
- **Quadratic time**: work that grows like n squared. It is called quadratic because the exponent is 2.
- **Dominant term**: the fastest-growing part of the expression. Big O keeps this part because it controls the behavior for large inputs.

Common Mistakes and Helpful Tips

- Mistake: thinking Big O means exact seconds. It does not; it describes growth pattern.
- Mistake: treating n like a fixed number. n means input size, so it changes with the problem.
- Mistake: assuming all loops are automatically bad. A single loop is usually normal linear work.
- Mistake: ignoring nested loops. A loop inside a loop over the same input often changes the shape dramatically.
- Tip: ask what happens when the input doubles. If work doubles, think O(n). If work quadruples, think O(n^2).
- Memory hook: Big O is not a stopwatch. Big O is a growth meter.

Notes

Made with Goodnotes

Lesson 8 - Analyzing Algorithms with Big O

Big Idea

Lesson 8 turns Big O from vocabulary into a reading skill. Instead of only recognizing the labels O(1), O(n), and O(n^2), you practice looking at code and deciding which growth pattern it matches.

The main move is to focus on the parts of the code that grow with the input. Fixed setup lines, fixed print statements, and constant multipliers matter in real programs, but they do not usually change the Big O class. Big O keeps the fastest-growing part because that is the part that dominates when n gets large.

Full Teaching Explanation

When you analyze code, do not start by counting every tiny operation forever. Start by identifying the shape. A plain statement like **total = 0** is O(1) because it runs once. A loop over a list is usually O(n) because it runs once for each item. A loop inside another loop is often O(n^2) because the inner loop repeats for every pass of the outer loop.

The course examples use simple step counts to build intuition. If a list has 3 names, a loop runs 3 times. If the list grows to 6 names, the loop runs 6 times. That is linear growth. But if each item has to be compared with every other item, the work grows much faster because every new item creates many new pair checks.

Big O also drops details that do not affect the long-term shape. In 5n + 7, the +7 is fixed extra work and the 5 is a constant multiplier, so the expression becomes O(n). In n^2 + 5n + 20, the n^2 term grows faster than the linear and constant parts, so the whole expression becomes O(n^2). This is the "dominant term" rule.

Big O Analysis Rules

Rule	What to Do	Example
Ignore constants	Remove fixed multipliers because they do not change the growth shape.	7n becomes O(n).
Drop lower-order terms	Keep the fastest-growing term and remove the smaller ones.	n^2 + n becomes O(n^2).
Single loop	If the loop runs once per item, it is usually linear.	for item in items: print(item) is O(n).
Sequential loops	If loops happen one after another, add them first, then simplify.	O(n) + O(n) becomes O(n).
Nested loops	If one loop runs inside another over the same input, multiply the work.	n * n becomes O(n^2).
Worst-case thinking	When unsure, describe the upper bound of work the algorithm might need.	Searching may need the whole list if the item is last.

Code Example 1 - Fixed Work Is O(1)

```
print("hello")
print("hey")
print("again")
```

Line-by-line explanation: these three print lines run one time each. They do not depend on n. Whether your data set has 10 items or 10,000 items, this exact code still runs three print statements, so it is O(1).

Code Example 2 - One Loop Is O(n)

```
def print_items(items):
    for item in items:
        print(item)
```

Line-by-line explanation: the loop runs once per item. If items has 5 values, print runs 5 times. If items has 500 values, print runs 500 times. The work grows with n, so the function is O(n).

Made with Goodnotes

Code Example 3 - Two Separate Loops Are Still O(n)

```
def print_twice(items):
    for item in items:
        print(item)

    for item in items:
        print(item)
```

Line-by-line explanation: the first loop is n steps and the second loop is n steps, so the raw count is 2n. Big O drops the constant multiplier 2, leaving O(n). The code does twice as much work in real life, but the growth shape is still linear.

Code Example 4 - Nested Loops Are O(n^2)

```
def print_pairs(items):
    for first in items:
        for second in items:
            print(first, second)
```

Line-by-line explanation: for each value in the outer loop, the inner loop runs through the whole list. If n is 3, that is 9 pair checks. If n is 10, that is 100 pair checks. The total grows like n * n, so the function is O(n^2).

Code Shape Quick Reference

Code Shape	How to Read It	Big O
A few fixed lines	Runs the same number of steps no matter input size.	O(1)
One loop over a list	Runs once for each item.	O(n)
Two loops one after another	Runs n + n steps, then constants are dropped.	O(n)
Loop inside a loop	Runs n times n steps.	O(n^2)
Expression 5n + 7	Drop the 5 and 7, keep n.	O(n)
Expression 2n^2 + 4n + 100	Drop constants and lower-order terms, keep n^2.	O(n^2)

Terms, Definitions, and Why They Are Called That

- **Analysis**: the process of breaking code down to understand how much work it does. It is called analysis because you are examining the structure, not just running it.
- **Step count**: a simplified count of operations. It helps reveal whether work grows slowly or quickly.
- **Constant**: a fixed number that does not change with input size, like 7 or 100.
- **Lower-order term**: a slower-growing part of an expression, like n inside n^2 + n.
- **Nested loop**: a loop inside another loop. It is called nested because one structure sits inside the other, like boxes inside boxes.
- **Dominant term**: the term that eventually matters most as n gets large. It dominates because it outgrows the others.

Common Mistakes and Debug Thinking

- Mistake: keeping every constant in the final Big O answer. Big O simplifies growth classes.
- Mistake: saying two separate loops are O(n^2). Separate loops add; nested loops multiply.
- Mistake: assuming every nested loop is automatically bad. Sometimes the input is small enough, but the growth pattern is still important.
- Mistake: choosing O(1) just because the code looks short. If a loop depends on input size, it grows with n.
- Debug tip: before choosing the Big O, mark which lines run once, which lines run once per item, and which lines run once per pair.
- Memory hook: code shape first, exact count second, dominant term last.

Made with Goodnotes

Mini Practice - Name the Growth

Question	Reasoning	Answer
for x in items: print(x)	One pass through the collection.	O(n)
print("start"); print("done")	Fixed number of steps.	O(1)
for a in items: for b in items: print(a, b)	Every item pairs with every item.	O(n^2)
3n + 20	Drop constant multiplier and fixed extra work.	O(n)
n^2 + 4n + 100	n^2 grows fastest.	O(n^2)

Notes

Made with Goodnotes

Lesson 9 - Challenge: Algorithm Analysis

Big Idea

This challenge is about proving that you can look at small Python snippets and name the growth pattern. Instead of only asking "what does this code print," the challenge asks, "how much work does this code do as the input gets bigger?" That is the difference between running code and analyzing code.

Full Teaching Explanation

Algorithm analysis is the skill of looking beneath the surface of code. Two programs can both be correct, but one may stay fast as the list grows while the other becomes painfully slow. In this lesson, you practice recognizing the common shapes: fixed work, one pass through a list, nested passes, and expressions where one term eventually dominates the others.

The most important habit is to identify what depends on the input size. If a line always runs once, it is constant work. If a loop runs once for every item in the list, the work grows with the list, so it is linear. If one loop runs inside another and both depend on the same input, the work usually grows like pairs of items, which is quadratic. If you see an expression like n^2 + 12n, the n^2 part eventually dominates, so the simplified Big O is O(n^2).

The challenge questions also teach you to avoid over-counting. Big O is not asking for the exact number of steps. It is asking for the growth shape. That means constants and smaller terms get dropped after you understand what they mean. This does not mean constants never matter in real apps; it means Big O is zoomed out enough to compare how algorithms scale.

Code Example - One Loop Is O(n)

```
items = [1, 2, 3, 4, 5]
for n in items:
    print(n)
```

Line-by-line explanation: the list has five values here, so the print statement runs five times. If the list had 500 values, it would run 500 times. The work grows in direct proportion to the number of items, so this is O(n).

Code Example - Fixed Prints Are O(1)

```
print("hello")
print("hello again")
print("one more hello")
```

Line-by-line explanation: there are exactly three print statements. That number does not change when input size changes, because there is no input-driven loop. Big O ignores the fixed constant 3, so this is O(1).

Made with Goodnotes

Code Example - Nested Loops Are O(n^2)

```
for i in range(n):
    for j in range(n):
        print(i, j)
```

Line-by-line explanation: for each value of i, the inner loop runs through every value of j. If n is 4, there are 16 prints. If n is 10, there are 100. That n times n pattern is O(n^2).

Challenge Pattern Match

Code Shape or Expression	Raw Thinking	Final Big O
One line that reads numbers[0]	Looks at one item no matter how long the list is.	O(1)
for item in items	One pass through all items.	O(n)
for i in items: for j in items	Every item is paired with every item.	O(n^2)
n^2 + 12n	n^2 grows faster than 12n as n gets large.	O(n^2)
7n + 30	Drop the constant multiplier and fixed add-on.	O(n)
Five print statements	Same amount of work every time.	O(1)

How to Answer These Questions Without Panicking

- **Step 1:** Find the part that repeats. If nothing repeats and no input-driven helper is hiding, it may be O(1).
- **Step 2:** Ask what controls the repeat count. A loop over a list usually depends on n.
- **Step 3:** Check whether loops are separate or nested. Separate loops add; nested loops multiply.
- **Step 4:** Simplify the expression. Drop constants and keep the fastest-growing term.
- **Step 5:** Match the shape: flat, straight line, curve, or explosion.

Common Mistakes and Helpful Tips

- **Mistake:** choosing O(1) just because the code looks short. A short loop can still run n times.
- **Mistake:** choosing O(n^2) for two loops that happen one after another. That is O(n) + O(n), which simplifies to O(n).
- **Mistake:** keeping n^2 + 12n as the final answer. In Big O, simplify to the dominant term.
- **Tip:** write a tiny input count: if n = 3, how many times does the key line run?
- **Memory hook:** count the repeating work, then simplify the growth.

Notes

Made with Goodnotes

Lesson 10 - Comparing Two Solutions to the Same Problem

Big Idea

This lesson shows why "it works" is not the finish line. Two solutions can produce the same answer, but one may be faster, cleaner, easier to explain, or safer to maintain. Algorithm analysis gives you a way to compare those choices instead of choosing based on vibes alone - although, tragically, vibes remain undefeated in group projects.

Full Teaching Explanation

The lesson compares solutions to the same task so you can see the trade-off between runtime and readability. One version may loop through a list once and keep a running total. Another version may use nested loops or unnecessary extra work. Both might eventually calculate the correct answer, but their growth patterns are different.

For small input, the difference may not feel dramatic. If a list has three items, O(n) and O(n^2) may both finish instantly. The difference appears when the input grows. With 1,000 items, an O(n) solution might do about 1,000 pieces of work, while an O(n^2) solution might do about 1,000,000 pair checks. That is why Big O matters before the code becomes painfully slow.

This lesson also adds nuance: faster is not the only measurement. If two solutions have the same Big O, the clearer one is usually better. Readability matters because future-you, classmates, teammates, and instructors need to understand the logic. A clever trick that technically works but nobody can explain is often worse than a straightforward solution with the same growth class.

Solution A - Clean Single Loop

```
def sum_list(nums):
    total = 0
    for n in nums:
        total = total + n
    return total
```

Line-by-line explanation: total starts at 0, then each item is added once. The loop runs n times, so this is O(n). It is also readable because the variable names match the task: sum the list.

Made with Goodnotes

Solution B - Nested Loop Version

```
def sum_list_slow(nums):
    total = 0
    for i in range(len(nums)):
        for j in range(len(nums)):
            total = total + nums[i]
    return total
```

Line-by-line explanation: the outer loop runs n times and the inner loop also runs n times for each outer step. That creates n * n work, or O(n^2). It is harder to read and does far more work than needed.

Comparing Two Correct Solutions

Question	Solution A: Single Loop	Solution B: Nested Loop
Does it solve the problem?	Yes, it can total the list.	It may produce a result, but it does unnecessary repeated work.
Growth pattern	O(n) - one pass.	O(n^2) - pair-style repeated work.
Readability	Clear: total each number once.	Confusing: why loop over every pair?
Best use	Beginner-friendly and efficient.	Mostly useful as a warning example.
Main lesson	Prefer clear, direct work.	Avoid repeated work unless the problem truly requires it.

Readability vs Speed

A solution can be faster but harder to read, or slower but easier to understand. The best answer depends on the situation. For beginner CS102 work, the first question is usually: can you explain it clearly? After that, ask whether the Big O is reasonable. If one solution is both clearer and faster, choose that one and give yourself a tiny parade.

If two solutions have the same Big O, readability may decide the winner. For example, a clean single loop and a clever loop trick might both be O(n). If the clean version is easier to understand and less likely to break, it is usually the better choice. Big O tells you about growth, but it does not replace judgment.

Same Big O, Different Readability

```
def count_evens_v1(nums):
    count = 0
    for n in nums:
        if n % 2 == 0:
            count = count + 1
    return count


def count_evens_v2(nums):
    return sum(1 for n in nums if n % 2 == 0)
```

Line-by-line explanation: both versions check each number once, so both are O(n). Version 1 is more beginner-readable because it shows the count changing step by step. Version 2 is shorter, but it hides more Python knowledge inside one line.

Made with Goodnotes

Decision Guide for Choosing a Solution

Decision Factor	Ask Yourself	What to Prefer
Correctness	Does it always produce the right answer?	Correct first. Fast wrong code is still wrong code.
Growth	How does work change as input grows?	Lower growth when input can get large.
Readability	Can I explain this to a beginner?	Clear code unless speed truly requires complexity.
Edge cases	Does it handle empty or weird input?	Safe code that does not crash unnecessarily.
Maintainability	Would future-me understand this next week?	Names and structure that reveal the logic.

Common Mistakes and Helpful Tips

- **Mistake:** assuming the shortest code is always the best code. Short can be elegant, but short can also be cryptic.
- **Mistake:** comparing exact steps only for tiny lists. Growth matters most when input gets bigger.
- **Mistake:** ignoring readability because one solution feels more advanced. Advanced code should still be explainable.
- **Tip:** when comparing two solutions, write: correctness, Big O, readability, edge cases.
- **Memory hook:** first make it right, then make it clear, then make it faster when needed.

Notes

Made with Goodnotes

Lesson 11 - Safe Algorithms: Handling Edge Cases and Bad Input

Big Idea

A safe algorithm does not only work when the input is perfect. It plans for edge cases: empty lists, missing values, zero denominators, weird types, or unexpected user input. This lesson is about writing code that fails less dramatically - because code throwing a tantrum is apparently not considered a feature.

Full Teaching Explanation

An edge case is a special situation at the edge of what your algorithm expects. If your function returns the first item in a list, the normal case is a list with at least one item. The edge case is an empty list. If your function divides two numbers, the normal case is a nonzero denominator. The edge case is division by zero.

Safe algorithms begin by checking dangerous conditions before they cause errors. These early checks are often called guards or guard clauses because they protect the rest of the code. A guard might say, "if the list is empty, return None," or "if the denominator is zero, return None." The point is not to ignore the problem; the point is to choose a safe, predictable response instead of letting Python crash with an error.

This matters because real programs rarely receive perfect input. Users leave forms blank. Lists are empty. Dictionaries are missing keys. Numbers arrive as strings. Files do not exist. APIs respond weirdly. Safe algorithms are more reliable because they expect imperfect conditions and handle them intentionally.

Unsafe Version - First Item Without a Guard

```
def first_item(numbers):
    return numbers[0]
```

Line-by-line explanation: this works only when numbers contains at least one item. If numbers is an empty list, numbers[0] causes an IndexError because there is no first position to access.

Safe Version - Guard Against an Empty List

```
def first_item(numbers):
    if len(numbers) == 0:
        return None
    return numbers[0]
```

Line-by-line explanation: the if statement checks the edge case first. If the list is empty, the function returns None safely. Otherwise, the function continues and returns the first item.

Made with Goodnotes

Safe Average - Guard Against Division by Zero

```
def average(numbers):
    if len(numbers) == 0:
        return None

    total = 0
    for n in numbers:
        total = total + n

    return total / len(numbers)
```

Line-by-line explanation: the guard handles the empty list before division happens. Without the guard, total / len(numbers) would become total / 0, which causes a division-by-zero error.

Common Edge Cases

Edge Case	Why It Is Risky	Safe Response
Empty list []	There may be no item at index 0 and no count for averages.	Return None, 0, or an empty result depending on the task.
One-item list	Loops and comparisons may assume multiple items exist.	Check whether the algorithm still makes sense for one value.
Missing dictionary key	Accessing data[key] can crash with KeyError.	Use .get(), check key membership, or provide a default.
Zero denominator	Division by zero causes an error.	Check if denominator == 0 before dividing.
Bad user input	A string like "cat" cannot become an integer.	Use validation or try/except.
None value	None is not the same as 0 or an empty string.	Check for None before math or string operations.

Safe Divide

```
def safe_divide(a, b):
    if b == 0:
        return None
    return a / b

print(safe_divide(10, 2))
print(safe_divide(10, 0))
```

Line-by-line explanation: the function checks b before dividing. When b is 0, it returns None instead of crashing. When b is 2, it returns 5.0.

Safe Dictionary Lookup

```
def get_user_name(user):
    if "name" not in user:
        return "Unknown"
    return user["name"]
```

Line-by-line explanation: the guard checks whether the dictionary contains the expected key. If not, the function returns a default value instead of causing a KeyError.

Made with Goodnotes

Safe User Input Conversion

```
def to_int(value):
    try:
        return int(value)
    except ValueError:
        return None
```

Line-by-line explanation: try/except is useful when an operation might fail even after you do your best to check it. If value can become an integer, the function returns the integer. If not, it returns None safely.

Terms, Definitions, and Why They Are Called That

- **Edge case:** an unusual but possible input near the boundary of normal expectations. It is called an edge case because it lives on the edge of the problem space.
- **Guard clause:** an early check that protects the rest of the function. It is called a guard because it stands at the entrance and blocks unsafe conditions.
- **Validation:** checking whether input is acceptable before using it.
- **Default value:** a backup value used when expected data is missing.
- **None:** Python value meaning "no value here." It is useful for safe failure when there is no meaningful answer.
- **Exception:** an error event Python raises when something goes wrong, such as ValueError, KeyError, or ZeroDivisionError.

Safe Algorithm Checklist

- What happens if the list is empty?
- What happens if there is only one item?
- What happens if a key is missing?
- What happens if a number is zero where division is needed?
- What happens if user input is the wrong type?
- What should the function return when it cannot produce a normal result?
- Can I explain the safe response clearly?

Notes

Made with Goodnotes

Code Example:



Made with Goodnotes

Lesson 12 - Challenge: Advanced Algorithm Analysis

Big Idea

This final lesson pulls the week together with a challenge-style review. You identify growth classes, simplify Big O expressions, recognize code shapes, and think about safe behavior. It is the boss battle: tiny Python snippets, big runtime feelings.

Full Teaching Explanation

By Lesson 12, you are expected to combine several skills at once. You should be able to look at a code snippet and ask: Does this run a fixed number of times? Does it loop once per item? Does it use a loop inside another loop? Does it double the number of possibilities each time? Then you should simplify the answer using Big O rules.

This lesson also reinforces that Big O is about growth, not exact timing. A function that checks only numbers[0] is O(1), because it looks at one item no matter how large the list is. A loop that checks every item is O(n), because the work grows with the number of items. A nested loop that compares every pair is O(n^2), because the work grows like n times n. A branching process that doubles possibilities can behave like O(2^n), which grows extremely fast.

The challenge also mixes in safe algorithm thinking. Advanced analysis is not only about speed. A good algorithm should be understandable, should handle edge cases, and should avoid unnecessary work. The strongest answer is often the one that is correct, clear, safe, and reasonably efficient.

Round 1 - Constant or Not?

```
def print_first(numbers):
    if len(numbers) > 0:
        print(numbers[0])
```

Answer: O(1). The function only checks the length and prints the first item if one exists. It does not scan the whole list, so the work does not grow with n.

Round 2 - One Loop

```
def print_all(numbers):
    for n in numbers:
        print(n)
```

Answer: O(n). The loop runs once for each number. If the list doubles, the number of print operations doubles too.

Made with Goodnotes

Round 3 - Nested Pair Checks

```
def print_pairs(numbers):
    for a in numbers:
        for b in numbers:
            print(a, b)
```

Answer: O(n^2). For every outer item, the inner loop runs through every item. This creates pair-style work: n times n.

Advanced Challenge Quick Reference

Pattern	Plain-English Meaning	Big O
Look at first item only	One check regardless of list length.	O(1)
Loop once over a list	One action per item.	O(n)
Two separate loops	n + n steps, simplified.	O(n)
Loop inside a loop	Each item pairs with each item.	O(n^2)
Expression 5n + 8	Drop constants and keep n.	O(n)
Expression n^2 + 3n + 20	n^2 grows fastest.	O(n^2)
Repeated doubling	Possibilities double each round.	O(2^n)

Boss Battle Strategy

- **Round 1:** Check whether the code only does fixed work. If yes, it is usually O(1).
- **Round 2:** Look for a single loop over the input. That usually gives O(n).
- **Round 3:** Look for nested loops. If both depend on n, think O(n^2).
- **Round 4:** Simplify expressions by dropping constants and lower-order terms.
- **Round 5:** Watch for doubling patterns, which can point toward O(2^n).
- **Round 6:** Do a safety check: empty input, missing values, bad values, and clear return behavior.

Mini Practice - Final Growth Check

Snippet or Expression	How to Think	Answer
numbers[0]	Only one item is accessed.	O(1)
for n in numbers	Loop runs once per item.	O(n)
for a in nums: for b in nums	Every item combines with every item.	O(n^2)
2n + 100	Constants do not change growth class.	O(n)
7n^2 + n + 4	n^2 dominates.	O(n^2)
Choices double every step	Work explodes as possibilities double.	O(2^n)

Made with Goodnotes

Common Mistakes and Helpful Tips

- **Mistake:** confusing O(1) with "one line of code." A one-line list operation can still hide more work.
- **Mistake:** thinking O(n) means slow. O(n) is often perfectly fine; it just grows with input size.
- **Mistake:** calling every loop O(n^2). One loop is O(n); nested loops are what usually create O(n^2).
- **Mistake:** forgetting safety while analyzing speed. Efficient code that crashes on empty input is not finished.
- **Tip:** when stuck, test tiny values: n = 1, n = 2, n = 3. Count the key operation.
- **Memory hook:** find the code shape, count the growth, simplify the answer, then check safety.

Final Week 3 Connection

Lessons 9-12 close the loop on the week. You started by learning that algorithms are clear steps. Then you learned to count those steps, describe how the work grows, compare two possible solutions, and make algorithms safer. That full chain is what makes programming feel less like guessing and more like structured problem solving.

When you return to this later, remember the flow: **write the steps, count the work, name the growth, compare choices, protect edge cases.** That is the Week 3 brain map.

Notes

Made with Goodnotes

Week 3 Review Tools - Algorithms and Big O

Big Picture Review

Week 3 is the bridge from 'I can write code' to 'I can reason about code.' The key skill is learning to describe an algorithm clearly, count the amount of work it does, and compare two possible solutions without getting hypnotized by small details.

For this week, do not memorize Big O like random alphabet soup. Treat it like a growth story: when the input gets bigger, does the work stay flat, grow one step at a time, grow in pairs, or explode into repeated doubling? That growth pattern is what tells you whether an algorithm will still feel reasonable when the data gets large.

Week 3 Master Map

Topic	What You Should Remember	Fast Example
Algorithm	A clear ordered set of steps used to solve a problem.	Order pizza, brush teeth, clean room
Pseudocode	Plain-English steps that plan the logic before Python.	Get plate -> add bread -> eat
Counting work	Count how many times important steps run.	A loop over 5 names prints 5 times
Complexity	How work grows when the input grows.	More contacts = more work
Big O	A simplified growth label for algorithm efficiency.	O(1), O(n), O(n^2)
Trade-off	Choosing between speed, clarity, safety, and maintainability.	Fast code vs readable code

One-Sentence Memory Hook

- Algorithm analysis means: write clear steps, count the growing work, simplify the growth class, compare choices, and protect the weird inputs.

Notes

Made with Goodnotes

Week 3 Glossary - Terms, Meanings, and Examples

How to Use This Glossary

Use this page like your translation dictionary. When a word looks fancy, translate it back into plain English first. Most Week 3 confusion comes from treating vocabulary like magic instead of remembering that every term is describing something ordinary: steps, repeated work, growth, or safety.

Core Vocabulary

Term	Definition and Why It Is Called That	Example
Algorithm	A clear set of ordered steps used to solve a problem. It is called an algorithm because it is a repeatable method, not a vague idea.	Brush teeth, make lunch, send a text
Step	One action inside an algorithm. A step should be small enough that a person or computer can follow it.	Open app; type message
Pseudocode	Code-like planning written in plain English. It is called pseudocode because it is not real Python yet, but it acts like a code blueprint.	IF hungry THEN eat
Input size / n	The amount of data the algorithm works on. It is often called n because n represents a variable count.	A list with n names
Step count	How many times a key operation runs. Helps reveal growth.	Loop through 6 names = 6 steps
Complexity	How the work grows as the problem gets bigger. It is not exact seconds; it is growth behavior.	A phone search getting slower
Big O	A simplified label for growth rate. The O stands for order: general size of growth.	One loop over n items = O(n)
Dominant term	The fastest-growing part of an expression. It dominates because it eventually matters most.	n^2 dominates 5n + 20
Constant	A fixed number that does not change with input size. Big O drops constants when describing growth.	3 print lines = O(1)
Edge case	A possible but unusual input near the boundary of normal use. It can break code if not protected.	An empty list passed in
Guard clause	An early check that protects the function. It guards the rest of the code from unsafe situations.	If list is empty: return None

Made with Goodnotes

Algorithm and Pseudocode Review

Full Teaching Review

An algorithm is not the same thing as code. Code is one way to express an algorithm, but the algorithm is the thinking underneath. This is why the course begins with everyday examples like ordering pizza, washing clothes, and making sandwiches. If you can break a real task into clear steps, you can eventually turn that same logic into Python.

Pseudocode is the middle step between natural English and Python. It lets you plan the logic without worrying about exact syntax yet. Good pseudocode should be clear, ordered, and small enough to translate. If one line secretly contains five actions, split it. A computer cannot infer what you meant the way a person might.

Algorithm vs Pseudocode vs Python

Term	What It Looks Like	What It Is For
Algorithm idea	Clean room / make sandwich / send text	Names the problem or task
Pseudocode	IF list is empty THEN return None	Plans logic before exact Python
Python	if len(items) == 0: return None	Runs the algorithm in a real language

Pseudocode Example - Safe First Item

```
Task: Safely get the first item from a list

IF the list is empty:
    return None
ELSE:
    return the first item
```

Real English: Before touching the list, check that something is there. If the list has items, take the first one. If not, return a safe value instead of crashing.

Common Mistakes and Fixes

- Mistake: writing a giant step like "do laundry." Fix: split it into load washer, add soap, start machine, move to dryer, fold.
- Mistake: jumping straight to Python before the logic is clear. Fix: write pseudocode first.
- Mistake: making pseudocode too vague. Fix: use tiny, ordered actions.

Made with Goodnotes

Counting Work and Big O Decision Guide

Full Teaching Review

Counting work means identifying the part of the algorithm that repeats or grows. You usually do not count every tiny machine operation. Instead, you focus on the key action: printing a name, checking an item, comparing a pair, adding a number, or processing one record.

Big O then turns that count into a growth category. If the key work stays the same no matter how much input exists, it is O(1). If the work goes one-for-one with the number of items, it is O(n). If each item gets compared with many other items, the work usually grows as O(n^2).

Big O Decision Guide

Question to Ask	What It Usually Means	Big O
Does the same amount of work happen no matter how big input is?	Fixed work	O(1)
Does one loop visit each item once?	One pass through input	O(n)
Are there two separate loops, one after another?	n + n, which simplifies	O(n)
Is one loop nested inside another over the same input?	n times n pair work	O(n^2)
Does input size double but work grows by one step?	Repeated halving / divide-and-conquer shape	O(log n)
Does every choice create more choices?	Exploding branching growth	O(2^n) or worse

Trace Example - Single Loop

```
names = ["Ana", "Ben", "Leo"]
steps = 0

for name in names:
    steps = steps + 1
    print(name)

print("Total steps:", steps)
```

Line-by-line: the loop visits each name once. Three names means three loop runs. Six names means six loop runs. Because the work grows one-for-one with the input size, this shape is O(n).

Made with Goodnotes

Big O Growth Classes - What They Feel Like

Full Teaching Review

Big O names the shape of the work. The shape matters more than the exact number of seconds. A program that changes five things may seem fast with five items, but it may become painfully slow if the pattern multiplies or doubles every time the input grows.

A helpful mental test: ask whether adding more input causes the work to stay the same, grow smoothly, grow by pairs, or explode. The more dramatic the growth curve, the more careful you need to be.

Growth Class Comparison

Big O	Plain-English Meaning	Common Code Shape	Tiny Example
O(1)	Flat work: input size does not change the amount of work.	Direct access / fixed steps	Turn on light
O(log n)	Work grows slowly because the problem shrinks each round.	Repeated halving	Binary-style search
O(n)	Straight line growth: one main pass over input.	One loop through n items	Read a list once
O(n log n)	More than one loop but usually much better than quadratic.	Efficient sorting-ish patterns	Many practical sorting algorithms
O(n^2)	Pair work: each item gets compared with many others.	Nested loops over same input	Handshake pairs
O(2^n)	Branching explosion: possibilities double.	Try every yes/no combination	Subset-style choices

Simplifying Expressions

Expression	What to Drop	Final Big O	Why
5n	Constant multiplier 5	O(n)	Five times straight-line is still linear
n + 20	Small fixed extra work	O(n)	The constant fades as n grows
n^2 + n	Lower-order term n	O(n^2)	n^2 eventually dominates n
7n^2 + 3n + 10	Drop constants and lower terms	O(n^2)	Fastest-growing part wins
20	Fixed work	O(1)	No input growth

Made with Goodnotes

Code Shape Practice - Trace Before You Guess

How to Practice

When a question asks for Big O, do not start by staring at the whole code like it owes you money. First find the repeated shape: one loop, two separate loops, nested loops, fixed print lines, or branching choices. Then count what grows with input and ignore the small constants.

Practice 1 - What Is the Growth?

```
# Practice 1 - What is the growth?
for name in names:
    print(name)
```

Answer: O(n). The function checks one name each loop. More names means more loop runs.

Practice 2 - What Is the Growth?

```
# Practice 2 - What is the growth?
print("start")
print("middle")
print("end")
```

Answer: O(1). The function prints three fixed lines. The number of input items does not change the work.

Practice 3 - What Is the Growth?

```
# Practice 3 - What is the growth?
for a in nums:
    for b in nums:
        print(a, b)
```

Answer: O(n^2). For every outer item, the inner loop runs across the whole list.

Trace Table - Tiny n Values

One Loop Runs	Nested Loop Work	What It Shows
1	1	Both are tiny
2	4	Nested work is already multiplying
10	100	Pairs grow quickly
100	10,000	Quadratic growth is much heavier
1,000	1,000,000	This is why Big O matters before production-size data

Made with Goodnotes

Comparing Solutions and Writing Safe Algorithms

Full Teaching Review

A correct answer is not automatically the best answer. Two algorithms can return the same result while having different speed, clarity, memory use, and safety. Week 3 asks you to keep thinking like a programmer who has to choose between options, not just make any code work once.

When two solutions have different Big O, the better-scaling option often matters more as input grows. When two solutions have the same Big O, readability becomes a major deciding factor. Safe algorithms also check bad or unusual inputs before those inputs crash the code.

Solution Comparison Checklist

Situation	Better Choice	Why
Need to count every item once	Single loop	Clear O(n) and easy to read
Nested loop does same job as one loop	Single loop	Avoids unnecessary O(n^2) work
Both are O(n), but one is clearer	Readable version	Humans maintain code later
Input might be empty	Guard clause	Prevents index/division errors
Input might be weird type	Validate before processing	Avoids confusing crashes

Safe First Item With Guard

```
def first_item(numbers):
    if len(numbers) == 0:
        return None
    return numbers[0]
```

The guard clause handles the empty-list edge case before the code touches index 0.

Edge Case Guard Guide

Risky Situation	Possible Guard
Empty list	if len(items) == 0: return None
Dividing by zero	if len(numbers) == 0: return None or message
Missing dictionary key	if key not in data: return fallback
Bad numeric input	try/validate range before math
None value	if value is None: return safe_default

Made with Goodnotes

Weekly Review Question Breakdown

How the Review Questions Think

The Week 3 review questions are not trying to make you memorize every screenshot. They are checking whether you can recognize the underlying pattern: clear steps, input size, repeated work, simplified Big O, comparison trade-offs, and safe edge-case handling.

Question Type Breakdown

Question Type	What It Is Really Asking	How to Answer
Best example of an algorithm	Which option has clear ordered steps?	Choose the list of actions, not a vague sentence or picture
What does n mean?	How many items does the input contain?	n = input size
Big O for a loop	Does work grow with n?	One loop over n items = O(n)
Big O for nested loops	Does one loop run inside another?	n times n = O(n^2)
Simplify expression	Which terms govern growth?	Drop constants and lower-order terms
Compare two solutions	Which is faster, clearer, or safer?	Mention speed and readability, not just code length
Edge case question	What happens with empty or bad input?	Look for a guard clause or safe return

Mini Answer Key Style Explanations

- Algorithm example: "open app -> enter PIN -> check balance" is an algorithm because it is ordered and repeatable.
- Three fixed print statements: O(1), because the number of prints does not depend on list size.
- One loop over a list: O(n), because work grows once for each item.
- Nested loops over the same list: O(n^2), because each outer item triggers a full inner pass.
- 5n + 20 simplifies to O(n), because constants and fixed additions do not change the growth shape.

Notes

Made with Goodnotes

Common Exam Traps and Debug Thinking

Trap Filter

The trap questions usually do not ask something wildly new. They take a familiar pattern and hide it inside extra words, constants, or a tiny code snippet. The goal is to ask whether you recognize the shape instead of panic-clicking whatever answer has the most intimidating parentheses.

Common Traps

Trap	Why It Tricks People	Correct Thinking
Three fixed print lines	People assume one line means O(1), three lines means O(3)	Still O(1), fixed work
Two separate loops	People see two loops and choose O(n^2)	Separate loops add: O(n)+O(n)=O(n)
Nested loops	People count only the visible outer loop	Inner loop runs for every outer item
Constants in expression	People keep 5n or 20 in answer	Drop constants and lower-order terms
Edge cases	Code works on normal input, so it seems safe	Test empty lists, zero, missing keys, and bad types
Faster but unreadable	People assume faster always wins	If Big O is same, readability may matter more

Debug Thinking Checklist

- What is the input size n?
- Which line or operation is repeated work?
- Does the code visit the same amount of work no matter what? If yes, think O(1).
- Is there one loop over input? Think O(n).
- Is there a loop inside another loop over input? Think O(n^2).
- Does the expression or code contain constants to drop?
- Does the function have a guard for empty or bad input?

Trap Example - Separate Loops Are Not Nested

```
# Separate loops are NOT nested
for x in nums:
    print(x)

for y in nums:
    print(y)
```

Answer: O(n). The loops are separate, so the work is n + n. Big O drops the constant multiplier and keeps O(n).

Made with Goodnotes

Week 3 Cheat Sheet

Fast Recall Table

Concept	Quick Version
Algorithm	Ordered steps to solve a problem
Good algorithm	Clear, ordered, repeatable, testable, and safe
Pseudocode	Plain-English logic before real syntax
Step count	How many times the key action runs
Input size n	How much data the algorithm receives
Complexity	How work grows when n grows
Big O	Simplified growth label
O(1)	Flat work
O(n)	One pass through input
O(n^2)	Nested pair-style work
Dominant term	Fastest-growing part of an expression
Guard clause	Early safety check
Edge case	Unusual but possible input that needs safe handling

Big O Naming Recipe

Step	Ask Yourself
1	What is n?
2	What line repeats?
3	How many times does it run?
4	Could two loops be separate or nested?
5	What is the raw expression?
6	Drop constants and lower-order terms
7	Name the Big O growth class
8	Check edge cases and readability before calling it done

Three Shapes to Memorize

```
O(1)
print("hello")

O(n)
for item in items:
    print(item)

O(n^2)
for a in items:
    for b in items:
        print(a, b)
```

Real English: fixed work, one pass, and pair work. If you can spot those three, most Week 3 questions get much less rude.

Made with Goodnotes

Final Week 3 Memory Sheet

The Whole Week in One Teaching Flow

An algorithm starts as a plan: a clear set of steps that transforms a starting situation into a finished result. Pseudocode lets you write that plan without worrying about Python syntax yet. Once the plan exists, you can count how much work it does. If the work is fixed, it is constant. If the work grows with every item, it is linear. If every item gets paired with every other item, it is quadratic.

Big O is the language for naming those growth shapes. It does not care about exact seconds or exact tiny operations. It cares about how the work grows as the input gets bigger. That is why constants and smaller terms usually disappear. What matters most is the dominant shape.

The final layer is judgment. A better solution is not only one that works. It should be clear enough to understand, efficient enough for the input size, and safe enough to handle edge cases. That is the Week 3 mindset: clear steps, counted work, named growth, thoughtful choices, safe behavior.

Final Self-Check Questions

- Can I explain what an algorithm is without using code?
- Can I turn an everyday task into ordered pseudocode?
- Can I identify the input size n?
- Can I count repeated work in a loop?
- Can I tell the difference between separate loops and nested loops?
- Can I simplify 7n^2 + 20n + 10?
- Can I compare two solutions by speed and readability?
- Can I add a guard clause for an empty list or zero divisor?
- Can I explain my Big O answer in plain English?

Last-Minute Memory Hooks

Phrase	Meaning
Clear steps first	Do not code a plan you cannot explain
Count the repeated work	Big O usually comes from what repeats
Separate loops add	n + n becomes O(n)
Nested loops multiply	n x n becomes O(n^2)
Fastest term wins	n^2 beats n; n beats 20
Guard the edge	Handle empty, missing, zero, and weird input before it crashes

Made with Goodnotes

Final Notes and Questions

Review Reflection Prompt

Use this page for anything you want to ask, rewrite in your own words, or practice again later. The strongest review habit is to turn confusing summaries back into full explanations: What is the input? What repeats? What grows? What gets ignored? What edge case could break this?

Questions / Extra Practice Notes

Made with Goodnotes